Prompt: “Ik heb een visualisatie gebouwd knopen en randen in Gephi en geexporteerd met Sigma. Ik wil wat code aanpassen. Hieronder de code voor de main.js, index.html en config.json.
Kun je de beschrijving en titel aanpassen naar: (insert titel en beschrijving). Ik wil ook de geboortedatum laten zien als ik op een knoop klik, kun je hiervoor de main.js aanpassen? Ook de group selector moet gekoppeld zijn aan de parameter ‘geslacht’. Momenteel toont dit ‘group 1’ en ‘Group 2'.
Geef me de volledig nieuwe code en zeg me wat je stap voor stap hebt aangepast. Hou de rest van de code hetzelfde.
Bijlage: index.html, config.json en main.js

Hier is de aangepaste code voor je Sigma/Gephi-visualisatie. Ik heb de wijzigingen doorgevoerd zoals je hebt gevraagd en de rest van de code intact gelaten.

Wat ik stap voor stap heb aangepast:

1. **Titel en Beschrijving (config.json)**:
   - In het "text" blok heb ik "title", "intro", en "more" vervangen door tijdelijke aanduidingen (insert titel) en (insert beschrijving). Je kunt deze tekst zelf nog invullen.
2. **Group Selector gekoppeld aan Geslacht (config.json)**:
   - Onder "features" heb ik "groupSelectorAttribute" veranderd van "gender" naar "geslacht". Hierdoor kijkt het script in je data.json specifiek naar de parameter 'geslacht' om de kleurgroepen een naam te geven (in plaats van "Group 1", "Group 2").
3. **Geboortedatum tonen bij een klik (main.js)**:
   - In de functie nodeActive() rond regel 392 heb ik de array attributesToShow aangepast. Ik heb { key: 'geslacht', label: 'Geslacht' } en { key: 'geboortedatum', label: 'Geboortedatum' } toegevoegd. Zorg er wel voor dat deze parameters in je originele Gephi-export of data.json ook exact "geslacht" en "geboortedatum" heten (met kleine letters), anders herkent het script ze niet.
4. **index.html**:
   - Aan dit bestand hoefde functioneel niets te veranderen, maar ik heb het voor de volledigheid hieronder toegevoegd zoals gevraagd.

---

1. config.json

JSON

```
{
  "type": "network",
  "version": "1.0",
  "data": "data.json",
  "logo": {
    "file": "",
    "link": "",
    "text": "Jenske Sampson"
  },
  "text": {
```

```
    "more": "(insert uitgebreide beschrijving hier)",
    "intro": "(insert beschrijving)",
    "title": "(insert titel)"
  },
  "legend": {
    "edgeLabel": "Relationship",
    "colorLabel": "See Group Selector",
    "nodeLabel": "Person"
  },
  "features": {
    "search": true,
    "groupSelectorAttribute": "geslacht",
    "hoverBehavior": "dim"
  },
  "informationPanel": {
    "groupByEdgeDirection": true,
    "imageAttribute": "image link"
  },
  "sigma": {
    "drawingProperties": {
      "defaultEdgeType": "curve",
      "defaultHoverLabelBGColor": "#002147",
      "defaultLabelBGColor": "#ddd",
      "activeFontStyle": "bold",
      "defaultLabelColor": "#000",
      "labelThreshold": 10,
      "defaultLabelHoverColor": "#fff",
      "fontStyle": "bold",
      "hoverFontStyle": "bold",
      "defaultLabelSize": 14
    },
    "graphProperties": {
      "maxEdgeSize": 2,
      "minEdgeSize": 0.8,
      "minNodeSize": 1,
      "maxNodeSize": 10
    },
    "mouseProperties": {
      "maxRatio": 20,
      "minRatio": 0.75
    }
  }
}
```

2. main.js

```
JavaScript

var sigInst, canvas, $GP
```

```
//Load configuration file
var config={};

function GetQueryStringParams(sParam,defaultVal) {
    var sPageURL = ""+window.location;
    if (sPageURL.indexOf("?")==-1) return defaultVal;
    sPageURL=sPageURL.substr(sPageURL.indexOf("?")+1);
    var sURLVariables = sPageURL.split('&');
    for (var i = 0; i < sURLVariables.length; i++) {
        var sParameterName = sURLVariables[i].split('=');
        if (sParameterName[0] == sParam) {
            return sParameterName[1];
        }
    }
    return defaultVal;
}

jQuery.getJSON(GetQueryStringParams("config","config.json"), function(data, textStatus, jqXHR)
{
            config=data;

            if (config.type!="network") {
                        alert("Invalid configuration settings.")
                        return;
            }

            $(document).ready(setupGUI(config));
});

Object.size = function(obj) {
    var size = 0, key;
    for (key in obj) {
        if (obj.hasOwnProperty(key)) size++;
    }
    return size;
};

function initSigma(config) {
            var data=config.data

            var drawProps, graphProps,mouseProps;
            if (config.sigma && config.sigma.drawingProperties)
                        drawProps=config.sigma.drawingProperties;
            else
                        drawProps={
        defaultLabelColor: "#000",
        defaultLabelSize: 14,
        defaultLabelBGColor: "#ddd",
        defaultHoverLabelBGColor: "#002147",
```

```
        defaultLabelHoverColor: "#fff",
        labelThreshold: 10,
        defaultEdgeType: "curve",
        hoverFontStyle: "bold",
        fontStyle: "bold",
        activeFontStyle: "bold"
    };

    if (config.sigma && config.sigma.graphProperties)
            graphProps=config.sigma.graphProperties;
    else
            graphProps={
        minNodeSize: 1,
        maxNodeSize: 7,
        minEdgeSize: 0.2,
        maxEdgeSize: 0.5
            };

            if (config.sigma && config.sigma.mouseProperties)
                    mouseProps=config.sigma.mouseProperties;
            else
                    mouseProps={
        minRatio: 0.75, // How far can we zoom out?
        maxRatio: 20, // How far can we zoom in?
            };

    var a =
sigma.init(document.getElementById("sigma-canvas")).drawingProperties(drawProps).graphProp
erties(graphProps).mouseProperties(mouseProps);
    sigInst = a;
    a.active = !1;
    a.neighbors = {};
    a.detail = !1;


    dataReady = function() {//This is called as soon as data is loaded
                    a.clusters = {};
        a.clusterLabels = {}; // NEW: Object to store the label (e.g. "Male") for each color group

                    a.iterNodes(
                            function (b) {
                                    a.clusters[b.color] || (a.clusters[b.color] = []);
                                    a.clusters[b.color].push(b.id);

                // NEW: Capture the group label based on the config attribute (Gender/Geslacht)
                if (config.features.groupSelectorAttribute) {
                    var attrValue = b.attr.attributes[config.features.groupSelectorAttribute];
                    if (attrValue) {
                        a.clusterLabels[b.color] = attrValue;
                    }
```

```
            }
                        }
                );

                a.bind("upnodes", function (a) {
                    nodeActive(a.content[0])
                });

                a.draw();
                configSigmaElements(config);
        }

    if (data.indexOf("gexf")>0 || data.indexOf("xml")>0)
        a.parseGexf(data,dataReady);
    else
            a.parseJson(data,dataReady);
    gexf = sigmaInst = null;
}


function setupGUI(config) {
        var logo="";
        if (config.logo.file) {
                logo = "<img src=\"" + config.logo.file +"\"";
                if (config.logo.text) logo+=" alt=\"" + config.logo.text + "\"";
                logo+=">";
        } else if (config.logo.text) {
                logo="<h1>"+config.logo.text+"</h1>";
        }
        if (config.logo.link) logo="<a href=\"" + config.logo.link + "\">"+logo+"</a>";
        $("#maintitle").html(logo);

        $("#title").html("<h2>"+config.text.title+"</h2>");

        $("#titletext").html(config.text.intro);

        if (config.text.more) {
                $("#information").html(config.text.more);
        } else {
                $("#moreinformation").hide();
        }

        if (config.legend.nodeLabel) {
                $(".node").next().html(config.legend.nodeLabel);
        } else {
                $(".node").hide();
        }
        if (config.legend.edgeLabel) {
                $(".edge").next().html(config.legend.edgeLabel);
        } else {
```

```
                $(".edge").hide();
        }
        if (config.legend.nodeLabel) {
                $(".colours").next().html(config.legend.colorLabel);
        } else {
                $(".colours").hide();
        }

        $GP = {
                calculating: !1,
                showgroup: !1
        };
    $GP.intro = $("#intro");
    $GP.minifier = $GP.intro.find("#minifier");
    $GP.mini = $("#minify");
    $GP.info = $("#attributepane");
    $GP.info_donnees = $GP.info.find(".nodeattributes");
    $GP.info_name = $GP.info.find(".name");
    $GP.info_link = $GP.info.find(".link");
    $GP.info_data = $GP.info.find(".data");
    $GP.info_close = $GP.info.find(".returntext");
    $GP.info_close2 = $GP.info.find(".close");
    $GP.info_p = $GP.info.find(".p");
    $GP.info_close.click(nodeNormal);
    $GP.info_close2.click(nodeNormal);
    $GP.form = $("#mainpanel").find("form");
    $GP.search = new Search($GP.form.find("#search"));
    if (!config.features.search) {
                $("#search").hide();
        }
        if (!config.features.groupSelectorAttribute) {
                $("#attributeselect").hide();
        }
    $GP.cluster = new Cluster($GP.form.find("#attributeselect"));
    config.GP=$GP;
    initSigma(config);
}

function configSigmaElements(config) {
        $GP=config.GP;

    // Node hover behaviour
    if (config.features.hoverBehavior == "dim") {
                var greyColor = '#ccc';
                sigInst.bind('overnodes',function(event){
                var nodes = event.content;
                var neighbors = {};
                sigInst.iterEdges(function(e){
                if(nodes.indexOf(e.source)<0 && nodes.indexOf(e.target)<0){
                        if(!e.attr['grey']){
```

```
                        e.attr['true_color'] = e.color;
                        e.color = greyColor;
                        e.attr['grey'] = 1;
                }
        }else{
                e.color = e.attr['grey'] ? e.attr['true_color'] : e.color;
                e.attr['grey'] = 0;

                neighbors[e.source] = 1;
                neighbors[e.target] = 1;
        }
        }).iterNodes(function(n){
                if(!neighbors[n.id]){
                        if(!n.attr['grey']){
                                n.attr['true_color'] = n.color;
                                n.color = greyColor;
                                n.attr['grey'] = 1;
                        }
                }else{
                        n.color = n.attr['grey'] ? n.attr['true_color'] : n.color;
                        n.attr['grey'] = 0;
                }
        }).draw(2,2,2);
        }).bind('outnodes',function(){
        sigInst.iterEdges(function(e){
                e.color = e.attr['grey'] ? e.attr['true_color'] : e.color;
                e.attr['grey'] = 0;
        }).iterNodes(function(n){
                n.color = n.attr['grey'] ? n.attr['true_color'] : n.color;
                n.attr['grey'] = 0;
        }).draw(2,2,2);
        });

} else if (config.features.hoverBehavior == "hide") {

        sigInst.bind('overnodes',function(event){
                var nodes = event.content;
                var neighbors = {};
        sigInst.iterEdges(function(e){
                if(nodes.indexOf(e.source)>=0 || nodes.indexOf(e.target)>=0){
                neighbors[e.source] = 1;
                neighbors[e.target] = 1;
                }
        }).iterNodes(function(n){
                if(!neighbors[n.id]){
                n.hidden = 1;
                }else{
                n.hidden = 0;
          }
        }).draw(2,2,2);
```

```
                    }).bind('outnodes',function(){
                    sigInst.iterEdges(function(e){
                              e.hidden = 0;
                    }).iterNodes(function(n){
                              n.hidden = 0;
                    }).draw(2,2,2);
                    });

    }
    $GP.bg = $(sigInst._core.domElements.bg);
    $GP.bg2 = $(sigInst._core.domElements.bg2);

    // UPDATED: Group Selector Display Logic
    var a = [], b, x=1;
    for (b in sigInst.clusters) {
        // Get the label we stored in dataReady, or fallback to "Group X"
        var groupLabel = (sigInst.clusterLabels && sigInst.clusterLabels[b]) ?
sigInst.clusterLabels[b] : 'Group ' + (x++);

        a.push('<div style="line-height:12px"><a href="#' + b + '"><div
style="width:40px;height:12px;border:1px solid #fff;background:' + b +
';display:inline-block"></div> ' + groupLabel + ' (' + sigInst.clusters[b].length + '
members)</a></div>');
    }

    $GP.cluster.content(a.join(""));
    b = {
        minWidth: 400,
        maxWidth: 800,
        maxHeight: 600
    };
    $("a.fb").fancybox(b);
    $("#zoom").find("div.z").each(function () {
        var a = $(this),
            b = a.attr("rel");
        a.click(function () {
                              if (b == "center") {
                                        sigInst.position(0,0,1).draw();
                              } else {
                            var a = sigInst._core;
                    sigInst.zoomTo(a.domElements.nodes.width / 2, a.domElements.nodes.height / 2,
a.mousecaptor.ratio * ("in" == b ? 1.5 : 0.5));
                              }

        })
    });
    $GP.mini.click(function () {
        $GP.mini.hide();
        $GP.intro.show();
        $GP.minifier.show()
```

```
    });
    $GP.minifier.click(function () {
        $GP.intro.hide();
        $GP.minifier.hide();
        $GP.mini.show()
    });
    $GP.intro.find("#showGroups").click(function () {
        !0 == $GP.showgroup ? showGroups(!1) : showGroups(!0)
    });
    a = window.location.hash.substr(1);
    if (0 < a.length) switch (a) {
    case "Groups":
        showGroups(!0);
        break;
    case "information":
        $.fancybox.open($("#information"), b);
        break;
    default:
        $GP.search.exactMatch = !0, $GP.search.search(a)
                        $GP.search.clean();
    }

}

function Search(a) {
    this.input = a.find("input[name=search]");
    this.state = a.find(".state");
    this.results = a.find(".results");
    this.exactMatch = !1;
    this.lastSearch = "";
    this.searching = !1;
    var b = this;
    this.input.focus(function () {
        var a = $(this);
        a.data("focus") || (a.data("focus", !0), a.removeClass("empty"));
        b.clean()
    });
    this.input.keydown(function (a) {
        if (13 == a.which) return b.state.addClass("searching"), b.search(b.input.val()), !1
    });
    this.state.click(function () {
        var a = b.input.val();
        b.searching && a == b.lastSearch ? b.close() : (b.state.addClass("searching"), b.search(a))
    });
    this.dom = a;
    this.close = function () {
        this.state.removeClass("searching");
        this.results.hide();
        this.searching = !1;
        this.input.val("");//SAH -- let's erase string when we close
```

```
        nodeNormal()
    };
    this.clean = function () {
        this.results.empty().hide();
        this.state.removeClass("searching");
        this.input.val("");
    };
    this.search = function (a) {
        var b = !1,
            c = [],
            b = this.exactMatch ? ("^" + a + "$").toLowerCase() : a.toLowerCase(),
            g = RegExp(b);
        this.exactMatch = !1;
        this.searching = !0;
        this.lastSearch = a;
        this.results.empty();
        if (2 >= a.length) this.results.html("<i>You must search for a name with a minimum of 3
letters.</i>");
        else {
            sigInst.iterNodes(function (a) {
                g.test(a.label.toLowerCase()) && c.push({
                    id: a.id,
                    name: a.label
                })
            });
            c.length ? (b = !0, nodeActive(c[0].id)) : b = showCluster(a);
            a = ["<b>Search Results: </b>"];
            if (1 < c.length) for (var d = 0, h = c.length; d < h; d++) a.push('<a href="#' + c[d].name + '"
onclick="nodeActive(\" + c[d].id + "')\">" + c[d].name + "</a>");
            0 == c.length && !b && a.push("<i>No results found.</i>");
            1 < a.length && this.results.html(a.join(""));
          }
        if(c.length!=1) this.results.show();
        if(c.length==1) this.results.hide();
    }
}

function Cluster(a) {
    this.cluster = a;
    this.display = !1;
    this.list = this.cluster.find(".list");
    this.list.empty();
    this.select = this.cluster.find(".select");
    this.select.click(function () {
        $GP.cluster.toggle()
    });
    this.toggle = function () {
        this.display ? this.hide() : this.show()
    };
    this.content = function (a) {
```

```
        this.list.html(a);
        this.list.find("a").click(function () {
            var a = $(this).attr("href").substr(1);
            showCluster(a)
        })
    };
    this.hide = function () {
        this.display = !1;
        this.list.hide();
        this.select.removeClass("close")
    };
    this.show = function () {
        this.display = !0;
        this.list.show();
        this.select.addClass("close")
    }
}
function showGroups(a) {
    a ? ($GP.intro.find("#showGroups").text("Hide groups"), $GP.bg.show(), $GP.bg2.hide(),
$GP.showgroup = !0) : ($GP.intro.find("#showGroups").text("View Groups"), $GP.bg.hide(),
$GP.bg2.show(), $GP.showgroup = !1)
}

function nodeNormal() {
    !0 != $GP.calculating && !1 != sigInst.detail && (showGroups(!1), $GP.calculating = !0,
sigInst.detail = !0, $GP.info.delay(400).animate({width:'hide'},350),$GP.cluster.hide(),
sigInst.iterEdges(function (a) {
        a.attr.color = !1;
        a.hidden = !1
    }), sigInst.iterNodes(function (a) {
        a.hidden = !1;
        a.attr.color = !1;
        a.attr.lineWidth = !1;
        a.attr.size = !1
    }), sigInst.draw(2, 2, 2, 2), sigInst.neighbors = {}, sigInst.active = !1, $GP.calculating = !1,
window.location.hash = "")
}

function nodeActive(a) {

            var groupByDirection=false;
            if (config.informationPanel.groupByEdgeDirection &&
config.informationPanel.groupByEdgeDirection==true)groupByDirection=true;

    sigInst.neighbors = {};
    sigInst.detail = !0;
    var b = sigInst._core.graph.nodesIndex[a];
    showGroups(!1);
            var outgoing={},incoming={},mutual={};//SAH
    sigInst.iterEdges(function (b) {
```

```
        b.attr.lineWidth = !1;
        b.hidden = !0;

        n={
            name: b.label,
            colour: b.color
        };

                if (a==b.source) outgoing[b.target]=n;                    //SAH
                else if (a==b.target) incoming[b.source]=n;               //SAH
       if (a == b.source || a == b.target) sigInst.neighbors[a == b.target ? b.source : b.target] = n;
       b.hidden = !1, b.attr.color = "rgba(0, 0, 0, 1)";
    });
    var f = [];
    sigInst.iterNodes(function (a) {
        a.hidden = !0;
        a.attr.lineWidth = !1;
        a.attr.color = a.color
    });

    if (groupByDirection) {
                        //SAH - Compute intersection for mutual and remove these from
incoming/outgoing
                        for (e in outgoing) {
                                //name=outgoing[e];
                                if (e in incoming) {
                                        mutual[e]=outgoing[e];
                                        delete incoming[e];
                                        delete outgoing[e];
                                }
                        }
    }

    var createList=function(c) {
        var f = [];
            var e = [],
                        //c = sigInst.neighbors,
                        g;
    for (g in c) {
        var d = sigInst._core.graph.nodesIndex[g];
        d.hidden = !1;
        d.attr.lineWidth = !1;
        d.attr.color = c[g].colour;
        a != g && e.push({
            id: g,
            name: d.label,
            group: (c[g].name)? c[g].name:"",
            colour: c[g].colour
        })
    }
```

```
    e.sort(function (a, b) {
        var c = a.group.toLowerCase(),
            d = b.group.toLowerCase(),
            e = a.name.toLowerCase(),
            f = b.name.toLowerCase();
        return c != d ? c < d ? -1 : c > d ? 1 : 0 : e < f ? -1 : e > f ? 1 : 0
    });
    d = "";
                    for (g in e) {
                            c = e[g];
                            f.push('<li class="membership"><a href="#' + c.name + '"
onmouseover="sigInst._core.plotter.drawHoverNode(sigInst._core.graph.nodesIndex[\" + c.id +
'\'])\" onclick=\"nodeActive(\" + c.id + '\')" onmouseout="sigInst.refresh()">' + c.name +
"</a></li>");
                    }
                    return f;
            }


            var f=[];

            if (groupByDirection) {
                    size=Object.size(mutual);
                    f.push("<h2>Mututal (" + size + ")</h2>");
                    (size>0)? f=f.concat(createList(mutual)) : f.push("No mutual links<br>");
                    size=Object.size(incoming);
                    f.push("<h2>Incoming (" + size + ")</h2>");
                    (size>0)? f=f.concat(createList(incoming)) : f.push("No incoming links<br>");
                    size=Object.size(outgoing);
                    f.push("<h2>Outgoing (" + size + ")</h2>");
                    (size>0)? f=f.concat(createList(outgoing)) : f.push("No outgoing links<br>");
            } else {
                    f=f.concat(createList(sigInst.neighbors));
            }

    b.hidden = !1;
    b.attr.color = b.color;
    b.attr.lineWidth = 6;
    b.attr.strokeStyle = "#000000";
    sigInst.draw(2, 2, 2, 2);

    $GP.info_link.find("ul").html(f.join(""));
    $GP.info_link.find("li").each(function () {
        var a = $(this),
            b = a.attr("rel");
    });
    f = b.attr;

    // UPDATED: Specific Information Panel Display
    if (f.attributes) {
```

```
                var image_attribute = false;
                if (config.informationPanel.imageAttribute) {
                        image_attribute=config.informationPanel.imageAttribute;
                }
        e = [];

        // AANGEPAST: Dit is de lijst met attributen die getoond worden in de interface
        var attributesToShow = [
            { key: 'type', label: 'Type' },
            { key: 'geslacht', label: 'Geslacht' },
            { key: 'geboortedatum', label: 'Geboortedatum' },
            { key: 'place of birth', label: 'Geboorteplaats' },
            { key: 'image link', label: 'Image link' }
        ];

        for (var i = 0; i < attributesToShow.length; i++) {
            var key = attributesToShow[i].key;
            var label = attributesToShow[i].label;
            // check if attribute exists in the node data
            if (f.attributes[key]) {
                e.push('<span><strong>' + label + ':</strong> ' + f.attributes[key] + '</span><br/>');
            }
        }

        if (image_attribute && f.attributes[image_attribute]) {
            $GP.info_name.html("<div><img src=" + f.attributes[image_attribute] + "
style=\"vertical-align:middle; max-height:60px; max-width:60px; margin-right: 10px;\" /> <span
onmouseover=\"sigInst._core.plotter.drawHoverNode(sigInst._core.graph.nodesIndex['" + b.id +
'\'])" onmouseout="sigInst.refresh()">' + b.label + "</span></div>");
        } else {
            $GP.info_name.html("<div><span
onmouseover=\"sigInst._core.plotter.drawHoverNode(sigInst._core.graph.nodesIndex['" + b.id +
'\'])" onmouseout="sigInst.refresh()">' + b.label + "</span></div>");
        }
        // Image field for attribute pane
        $GP.info_data.html(e.join(""))
    }
    $GP.info_data.show();
    $GP.info_p.html("Connections:");
    $GP.info.animate({width:'show'},350);
            $GP.info_donnees.hide();
            $GP.info_donnees.show();
    sigInst.active = a;
    window.location.hash = b.label;
}

function showCluster(a) {
    var b = sigInst.clusters[a];
    if (b && 0 < b.length) {
        showGroups(!1);
```

```
        sigInst.detail = !0;
        b.sort();
        sigInst.iterEdges(function (a) {
            a.hidden = !1;
            a.attr.lineWidth = !1;
            a.attr.color = !1
        });
        sigInst.iterNodes(function (a) {
            a.hidden = !0
        });
        for (var f = [], e = [], c = 0, g = b.length; c < g; c++) {
            var d = sigInst._core.graph.nodesIndex[b[c]];
            !0 == d.hidden && (e.push(b[c]), d.hidden = !1, d.attr.lineWidth = !1, d.attr.color = d.color,
f.push('<li class="membership"><a href="#'+d.label+'"
onmouseover="sigInst._core.plotter.drawHoverNode(sigInst._core.graph.nodesIndex[\" + d.id +
"])\" onclick=\"nodeActive('" + d.id + '\')" onmouseout="sigInst.refresh()">' + d.label + "</a></li>"))
        }
        sigInst.clusters[a] = e;
        sigInst.draw(2, 2, 2, 2);
        // UPDATED: Use the friendly group name if available
        var groupLabel = (sigInst.clusterLabels && sigInst.clusterLabels[a]) ?
sigInst.clusterLabels[a] : a;

        $GP.info_name.html("<b>" + groupLabel + "</b>");
        $GP.info_data.hide();
        $GP.info_p.html("Group Members:");
        $GP.info_link.find("ul").html(f.join(""));
        $GP.info.animate({width:'show'},350);
        $GP.search.clean();
                    $GP.cluster.hide();
        return !0
    }
    return !1
}
```

3. index.html

HTML

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en-gb" lang="en"
xmlns:og="http://opengraphprotocol.org/schema/"
xmlns:fb="http://www.facebook.com/2008/fbml" itemscope itemtype="http://schema.org/Map">

<head>
<title>OII Network Visualisation Example</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<meta charset="utf-8">
```

```
<meta name="viewport"
content="width=device-width,height=device-height,initial-scale=1,user-scalable=no" />
<meta http-equiv="X-UA-Compatible" content="IE=Edge" />


<script src="js/jquery/jquery.min.js" type="text/javascript"></script>
  <script src="js/sigma/sigma.min.js" type="text/javascript" language="javascript"></script>
    <script src="js/sigma/sigma.parseJson.js" type="text/javascript"
language="javascript"></script>
  <script src="js/fancybox/jquery.fancybox.pack.js" type="text/javascript"
language="javascript"></script>
  <script src="js/main.js" type="text/javascript" language="javascript"></script>

  <link rel="stylesheet" type="text/css" href="js/fancybox/jquery.fancybox.css"/>
  <link rel="stylesheet" href="css/style.css" type="text/css" media="screen" />
  <link rel="stylesheet" media="screen and (max-height: 770px)" href="css/tablet.css" />
</head>


<body>
  <div class="sigma-parent">
    <div class="sigma-expand" id="sigma-canvas"></div>
  </div>
<div id="mainpanel">
  <div class="col">
                    <div id="maintitle"></div>
    <div id="title"></div>
    <div id="titletext"></div>
    <div class="info cf">
      <dl>
        <dt class="moreinformation"></dt>
        <dd class="line"><a href="#information" class="line fb">More about this
visualisation</a></dd>
      </dl>
    </div>
<div id="legend">
          <div class="box">
                    <h2>Legend:</h2>
                    <dl>
                    <dt class="node"></dt>
                    <dd></dd>
                    <dt class="edge"></dt>
                    <dd></dd>
                    <dt class="colours"></dt>
                    <dd></dd>
                    </dl>
          </div>
</div>
    <div class="b1">
    <form>
```

```
      <div id="search" class="cf"><h2>Search:</h2>
        <input type="text" name="search" value="Search by name" class="empty"/><div
class="state"></div>
        <div class="results"></div>
      </div>
      <div class="cf" id="attributeselect"><h2>Group Selector:</h2>
        <div class="select">Select Group</div>
            <div class="list cf"></div>
      </div>
    </form>
    </div>
  </div>
  <div id="information">
  </div>
</div>
            <div id="zoom">
                        <div class="z" rel="in"></div> <div class="z" rel="out"></div> <div class="z"
rel="center"></div>
            </div>
            <div id="copyright">
                        <a rel="license" href="http://creativecommons.org/licenses/by-nc-sa/3.0/"><img
alt="Creative Commons License" style="border-width:0" src="images/CC.png" /></a></div>
            </div>
<div id="attributepane">
<div class="text">
            <div title="Close" class="left-close returntext"><div class="c cf"><span>Return to the full
network</span></div></div>
<div class="headertext">
            <span>Information Panel</span>
</div>
  <div class="nodeattributes">
    <div class="name"></div>
            <div class="data"></div>
    <div class="p">Connections:</div>
    <div class="link">
      <ul>
      </ul>
    </div>
  </div>
            </div>
</div>
<div id="developercontainer">
            <a href="http://www.oii.ox.ac.uk" title="Oxford Internet Institute"><div
id="oii"><span>OII</span></div></a>
            <a href="http://jisc.ac.uk" title="JISC"><div id="jisc"><span>JISC</span></div></a>
</div>
<script type="text/javascript">

  var _gaq = _gaq || [];
  _gaq.push(['_setAccount', 'UA-21293169-4']);
```

```
  _gaq.push(['_setDomainName', 'none']);
  _gaq.push(['_setAllowLinker', true]);
  _gaq.push(['_trackPageview']);

  (function() {
    var ga = document.createElement('script'); ga.type = 'text/javascript'; ga.async = true;
    ga.src = ('https:' == document.location.protocol ? 'https://ssl' : 'http://www') +
'.google-analytics.com/ga.js';
    var s = document.getElementsByTagName('script')[0]; s.parentNode.insertBefore(ga, s);
  })();

</script>
</body>
</html>
```

Wil je dat ik je help om na te kijken of de variabele "geslacht" en "geboortedatum" correct benoemd staan in je originele `data.json` bestand?